Titan Data Architecture & Table Doctrine

Purpose

This document defines the database architecture used by TitanZero.

The goal of this doctrine is to ensure:

predictable data structures

clear separation of platform and operational data

safe multi-tenant operation

scalable extension development

compatibility with AI reasoning systems

Every table in TitanZero must follow these structural rules.

Table Prefix Doctrine

TitanZero uses prefixes to clearly separate types of data.

tz_* → operational business data
ext_* → extension module data
(no prefix) → platform system tables

This separation prevents confusion between platform infrastructure and tenant-owned business data.

Platform Tables (No Prefix)

Platform tables represent global system infrastructure shared by all tenants.

Examples:

users
teams
team_users
roles
permissions
plans
menus
extensions

These tables manage:

authentication

tenancy

roles and permissions

system configuration

installed extensions

Platform tables typically do not include team_id, because they operate at the system level.

Operational Tables (tz_)

Tables prefixed with tz_ represent business data belonging to a tenant.

Examples:

tz_customers
tz_jobs
tz_job_visits
tz_tasks
tz_quotes
tz_invoices
tz_documents
tz_evidence_items

Rules for tz_ tables:

must include team_id

must follow lifecycle architecture

must emit signals on lifecycle transitions

must support audit history

Operational tables are the primary data used by the platform's engines.

Extension Tables (ext_)

Tables prefixed with ext_ belong to optional modules or extensions.

Examples:

ext_social_campaigns
ext_marketing_posts
ext_ai_content
ext_social_accounts

Rules for extension tables:

must include team_id

must not modify tz_ tables directly

must communicate with the system through signals and engine APIs

must remain isolated from the core schema

This allows extensions to be installed or removed without breaking the platform.

Tenant Isolation

TitanZero operates as a multi-tenant system.

Each tenant represents a company.

The tenant identifier is:

team_id

Rules:

every tz_ and ext_ table must include team_id

queries must always scope by team_id

engine APIs must enforce tenant boundaries automatically

This ensures that data belonging to one company cannot be accessed by another.

Core Entity Structure

Operational entities follow a consistent structure.

Example entity:

tz_jobs

Associated satellite tables:

tz_job_events
tz_job_notes
tz_job_links
tz_job_evidence

The pattern separates current state from lifecycle history.

Entity Table Types

Each entity typically includes several related tables.

Core Table

Contains current state.

Example:

tz_jobs

Fields may include:

id
team_id
customer_id
site_id
status
created_at
updated_at

Events Table

Records lifecycle changes.

Example:

tz_job_events

Fields:

id
team_id
job_id
event_type
payload
created_at

Events allow the system to reconstruct lifecycle history.

Notes Table

Stores human or system notes.

Example:

tz_job_notes

Fields:

id
team_id
job_id
note_text
created_by
created_at

Links Table

Stores relationships between entities.

Example:

tz_job_links

Links may connect jobs to:

documents

messages

evidence

invoices

Evidence Table

Stores references to captured evidence.

Example:

tz_job_evidence

Evidence itself is stored by the Titan Evidence Engine.

Event Sourcing Principles

TitanZero follows partial event-sourcing principles.

Key rules:

state changes generate events

events generate signals

signals drive automation

Example lifecycle:

job_started
↓
event recorded
↓
signal emitted
↓
automation triggered

This ensures that all actions remain traceable.

Signal Storage

Signals are stored in:

tz_signals

tz_signal_events
tz_signal_consumers

Signals represent the event stream of the system.

They are append-only and immutable.

Automation Storage

Automation rules and execution logs are stored in:

tz_automation_rules
tz_automation_triggers
tz_automation_runs
tz_automation_logs

These tables track automation behaviour and execution history.

Governance Storage

AI governance decisions are stored in tables such as:

tz_governance_decisions
tz_ai_requests
tz_ai_artifacts
tz_ai_routing_logs

These tables record:

assistant proposals

risk evaluations

approval outcomes

AI artifacts

This provides a complete audit trail for AI decisions.

Analytics Storage

Analytics data is stored separately from operational data.

Example tables:

tz_metrics
tz_metric_snapshots
tz_analytics_queries

Analytics engines consume signals rather than scanning operational tables directly.

Data Integrity Rules

Developers must follow these rules:

1. Do not bypass engine APIs when writing operational data.

2. Emit signals for all lifecycle transitions.

3. Do not delete operational records unless required by policy.

4. Preserve event history for auditability.

5. Enforce tenant isolation on every query.

These rules ensure data consistency and reliability.

Schema Evolution

When adding new entities or features:

follow prefix rules

follow entity satellite table pattern

register signals for lifecycle transitions

update automation and assistant subscriptions

This ensures the system remains extensible.

Summary

The Titan Data Architecture & Table Doctrine ensures that TitanZero's database remains structured, scalable, and safe for multi-tenant operation.

By separating platform tables, operational data, and extension data, and by following consistent entity structures with event history and signals, the system provides a reliable foundation for automation, AI reasoning, and analytics.